

Effective Library Design with PPR/Pt

Marc Dürner, Ph.D., 12.Oct.2006

NaviPlatform



MAP&GUIDE
The Mapware Company

Acknowledgements

PTV AG

especially:

Björn Streule

Christoph Poggemann

Hendrik Bock

Laurentiu Crisan

Michael Wohlwend

Olcay Gunes

Roman Schnider

Sebastian Pieck

Stefan Büder

Sven Falk

Tobias Müller

Open Source

Aloysius Indrayanto

Bernhard Rosenkränzer

Mike Sharov

Stephan Beal

Tommi Mäkitalo

Overview

C++ features

Techniques and principles

Module overview

API details of selected parts

Examples

Disclaimer

Hereby, I assume we know everything about C++.

... we may not.

***When you see or hear something you do not understand,
please ask !***

What are PPR and Platinum

Platinum C++ Framework (Pt)

- > Completely open source (LGPL)
- > <http://pt-framework.sourceforge.net/>
- > <https://svn.sourceforge.net/svnroot/pt-framework>
- > Everyone is invited to join !

PTV Portable Runtime (PPR)

- > Mostly open source (LGPL)
- > Adapted slightly to fit PTV's requirements
- > www.ptv.de

Portability

Tooling as important as Code !

- > Portable code starts with the build system...
- > ... ends with the tests !

Design Question

- > Layering / encapsulation
- > Emulation
- > Use standards !!!

Portability – Pimpl my Code (1)

```
class MutexImpl;

class Mutex {
public:
    Mutex();
    ~Mutex();
    lock();
    unlock();

private:
    MutexImpl* _impl;
};
```

```
#include „MutexImpl.h“

Mutex::Mutex()
{
    _impl = new MutexImpl();
}

Mutex::~Mutex()
{
    delete _impl;
}

Mutex::lock()
{ _impl->lock(); }

Mutex::unlock()
{ _impl->unlock(); }
```

Portability – Pimpl my Code (2)

/posix/MutexImpl.h

```
class MutexImpl {  
public:  
    Mutex()  
    { /* posix code */ }  
  
    ~Mutex()  
    { /* posix code */ }  
  
    lock();  
    { /* posix code */ }  
  
    unlock()  
    { /* posix code */ }  
};
```

/win32/MutexImpl.h

```
class MutexImpl {  
public:  
    Mutex()  
    { /* win32 code */ }  
  
    ~Mutex()  
    { /* win32 code */ }  
  
    lock();  
    { /* win32 code */ }  
  
    unlock()  
    { /* win32 code */ }  
};
```

Portability – Pimpl my Code (3)

MutexImpl is opaque to clients

> Clients are shielded from platform headers

Build system selects correct MutexImpl

No #ifdefs at all !!!

No virtual functions in MutexImpl interface

Fast Pimpl's get rid of new / delete

Flexibility

Prefer non-intrusive design

- > Can not always implement an interface

Make classes configurable

- > Policy-based design
- > Allocators

Exception safety

- > Others may throw even if we do not

Thread-safety

- > At what level do we lock?
- > Most parts can and should be reentrant

Flexibility (2)

Meyers Singleton

```
template <class T>
class Singleton
{
    public:
        static T& instance()
        {
            static T obj;           // initialised when first called
            return obj;             // dtor cleans up on program exit
        }

        // private ctor, copy ctor, dtor
        // and assignment operator
};
```

Flexibility (3)

A more flexible singleton using an allocator

```
template <class T, typename Allocator = std::allocator<T> >
class Singleton : public NonCopyable           // disallow copying
{
    static Allocator _alloc;
    static T* _instance;

public:
    static T& instance() {
        if ( !_instance ) {
            _instance = _alloc.allocate(1);           // get memory for 1 object
            new (_instance) T();                       // placement new
            std::atexit(&atExit);                       // clean up on program exit
        }

        return *_instance;
    }
}
```

Time and Space Constraints

Right mix of virtual inheritance and templates

- > Virtual inheritance small(er)
- > Templates faster (inlining crucial)

C++ can be more efficient than C

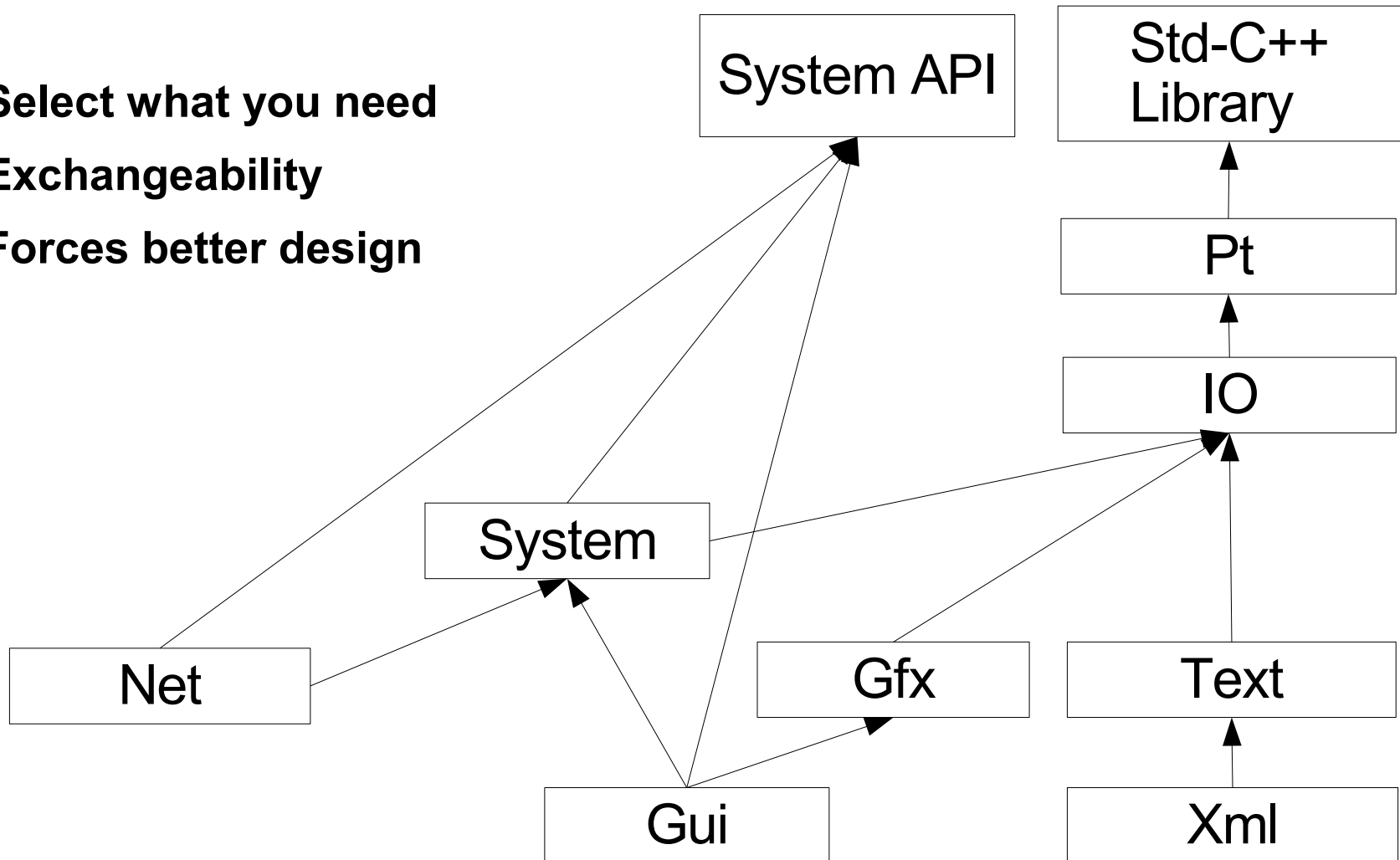
- > No-Cost features (Class Mechanics, Virtual Functions, new/delete)
- > Low-Cost features (Exceptions)
- > High-Cost features (RTTI, dynamic_cast)

Standards

- > Standard C++ library often optimized by vendor
- > std::fstream memory mapped
- > Type traits

Modularity

Select what you need
Exchangeability
Forces better design



Core Module Overview

Basic Types

- > POD's uint8_t ... int64_t
- > Time, Date, DateTime
- > AtomicInt

Error handling

- > Exceptions
- > SourceInfo

Object Modelling

- > Signals/Delegates
- > Singleton, NonCopyable...
- > SmartPtr
- > CTTI (Compile Time Type Information)
- > Reflection

Signals and Delegates (1)

Motivating Example: Write a Button class for a GUI toolkit

Try 1: Button class using virtual inheritance

```
class Button {  
    public:  
        virtual void onClick() = 0;           // to be implemented  
};
```

```
class MyButton {  
    public:  
        virtual void onClick();               // code to react on clicks  
};
```

Signals and Delegates (2)

Disadvantages:

- > Leads to many derived Button classes
- > Does not reflect one-to-many relationship well
- > Button has intimate knowledge of „click-code“

Signals and Delegates (3)

Try 2: Button class with listeners

```
class Listener {  
    public:  
        virtual void onClick() = 0;           // derive and implement  
};  
  
class Button {  
    std::list<Listener*> _listeners;  
  
    public:  
        void addListener( Listener& l);      // register derived listener  
        void onClick();                      // calls all registered listeners  
};
```

Signals and Delegates (4)

Advantages:

- > One-to-many relationship
- > Decouple sender from listener
- > Classic OO design

Disadvantages:

- > Intrusive Design
 - > What if we can not or do not want to derive from a Listener?
- > Interface Pollution
 - > Can I really have a onClick() function in my interface?
- > Still coupled to Listener Interface
 - > I already derive from LabelListener, isn't that enough?

Signals and Delegates (5)

Try 3: Button class with a signal

```
class Button {  
    public:  
        Signal<> clicked;  
} btn;
```

```
class View : public Connectable {  
    public:  
        int viewFunc();  
        static bool statFunc();  
        Signal<> dereferClick;  
} view;
```

```
void freeFunc();
```

```
connect(btn.clicked, view, &View::viewFunc);  
connect(btn.clicked, &View::statFunc);  
connect(btn.clicked, freeFunc);  
connect(btn.clicked, view.dereferClick);
```

Signals and Delegates (6)

Advantages:

- > Loose coupling, extremely flexible
 - > Connectable can work with all types of Signals or Delegates
- > Non-intrusive
 - > Connectable optional for Connection management
- > No interface pollution
- > One-to-many relationship and one-to-one relationship
 - > Signals vs Delegates
- > Very easy to use
 - > Compile time safety, Connection management
- > Binding of Signatures

Signal Syntax

Signals can have arguments:

- > Signal<> Signal with no arguments
- > Signal< int, int > Signal to callees taking two ints
- > Signal< const std::string& > Signal to callees taking a const std::string&

Invoking Signals

Signal< const std::string& > stringSignal;

[... code ...]

```
stringSignal(„Hello“);           // operator() of Signal
stringSignal.send(„Hello“);       // Signal::send
```

Connecting Signals

Establishing a connection

> Callee's signature must match signal's signature, except for return value

```
class Receiver : public Connectable
{
    public:
        void func();
};
```

```
Receiver rec;
Signal<> signal;
connect( signal, rec, &Receiver::func );
```

Using Connections

Connections

- > Can be closed
- > Are refcounted

```
class Receiver : public Connectable
{
    public:
        void func();
};
```

```
Receiver rec;
```

```
Signal<> signal;
```

```
Connection c = connect( signal, rec, &Receiver::func );
```

```
c.close();
```

```
signal.send(); // will not call rec
```

Connection Management

Auto Disconnect on destruction

- > When a connected object goes out of scope, its connections are automatically closed

```
class Receiver : public Connectable
{
    public:
        void func();
};
```

```
Receiver* rec = new Receiver() ;
Signal<> signal;
connect( signal, *rec, &Receiver::func );
delete rec;
```

```
signal.send(); // will not try to call rec anymore
```

Signals vs Delegates

Signals:

- > One-to-many
- > Ignore return value
- > Slightly slower than delegates
- > Slightly bigger than delegates

Delegates:

- > One-to-one
- > Return value forwarded
- > Slightly faster than signals
- > Slightly smaller than signals

Costs of Signals and Delegates

Signals:

- > 3 times slower than virtual function call
- > Contains Connection objects

Delegates:

- > As fast as virtual function call
- > Contains Connection objects

Costs are same as antique Listener concept

Qt Signals 10 times slower than C++ virtual function call

.Net Delegates 10 times slower than C# virtual function call

I/O Module Overview

Transparent Low-level I/O

- > IODevice
- > Implemented in higher-level modules

Transparent Buffered I/O

- > std::streambuf
- > Better peeking

Transparent I/O Streams

- > Real std::iostreams
- > Full support for l10n and i18n

Non-blocking I/O

System Module Overview

Filesystem handling

- > File, Directory ...

File I/O

- > Full IOStream support

Multithreading and processing

- > Thread, Process
- > Synchronisation: Spinlock, Mutex, Condition, Semaphore

Event handling

- > EventLoop, Application

SharedLibrary

Plugins

Multithreading

Resources used more efficiently

- > I/O time vs CPU time

Parallel execution

- > Sometimes we do not want to block
- > Asynchronous API without state-machine

Synchronisation overhead

- > No Synchronisation needed in single-threaded applications

Hard to debug

- > Design, design, design !!!

Living in a multithreaded world (1)

Assume, the variable `i` is an int.

Can you spot the bug in the following code?

```
++i;
```

Living in a multithreaded world (2)

What ++i really does (Read-Modify-Write):

- > Fetch value of i from memory into a register
 - > We might be preempted here
- > Increment value in register
 - > We might be preempted here
- > Put value back to memory
 - > We might be preempted here

In multithreaded code we can make much less assumptions about when code is executed

Solution:

- > AtomicInt uses CPU specific asm to achieve thread-safety

Levels of Thread-Safety

Thread-safe

- > A thread-safe function can be called from multiple threads.

Reentrant

- > Function can be called by multiple threads, as long as all callers work with unique data
- > Most C++ member functions are reentrant since they only work on member data

Thread-unsafe

- > Function accesses unprotected shared data.

Synchronisation Strategies

When should I lock?

- > If my objects and functions are reentrant they can be protected when used in a multi-threaded context.
- > High-level locking means no overhead for single-threaded programs

What should I lock

- > Multiple threads can read shared data if none writes at the same time
- > Constructors may be thread-safe

How should I lock?

- > Spinlock for short locks
- > Mutex for long locks
- > Condition and Semaphores to implement high-level locking strategies

Event Driven Applications

Event driven programs allow other threads to exist well separated from the main thread.

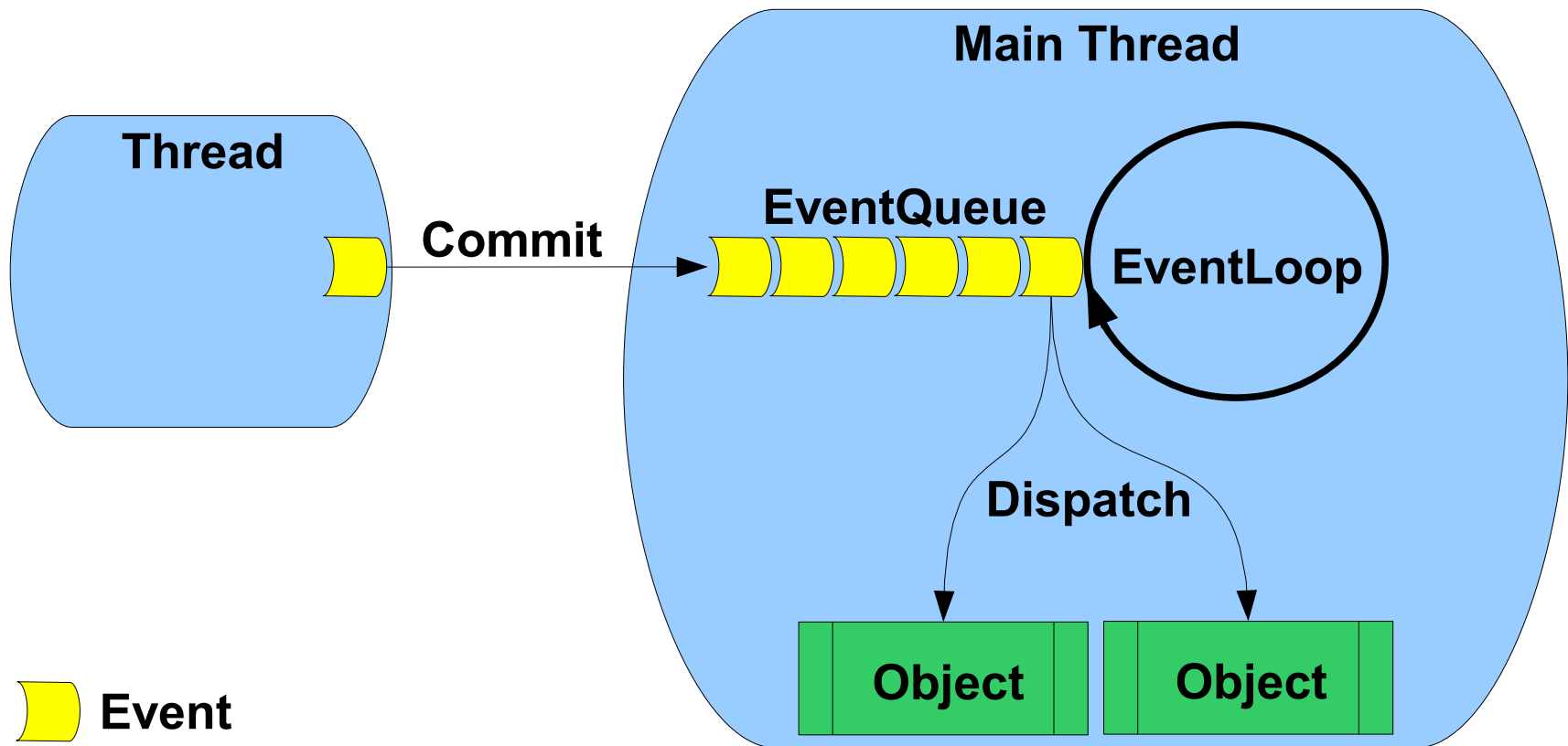
Eventloop is the central part of the application

- > Main thread handles events in the eventloop
- > If no events are present, main thread sleeps
- > If events are present, they are dispatched one at a time.

Other threads produce events

- > Produced events are committed to the eventloop
- > Threads can sleep while waiting for work

EventLoop Mechanics



A Multithreaded, Event-Driven „Hello World“ Application

Step 1: A custom event class

```
class HelloEvent : public ptv::Event {           // derive from ptv::Event
public:
    HelloEvent(const std::string& text)
        : _text(text)
    {}

    static const std::type_info& TypeInfo;       // a static type info

    ptv::Event* clone() const                   // implement cloning
    { return new HelloEvent(*this); }

    const std::type_info& typeId() const        // implement rtti
    { return HelloEvent::TypeInfo; }

    const std::string& text() const
    { return _text; }

};
```

A Multithreaded, Event-Driven „Hello World“ Application

Step 2: A producer thread

```
class HelloThread : public ptv::system::Thread {  
    public:  
        virtual void run()                // implement thread function  
        {  
            while( true ) {  
                system::Thread::sleep(1000);  
                HelloEvent ev(„Hello World“);  
                greet.send( ev );          // send new event  
            }  
        }  
  
        ptv::Signal<const ptv::Event&> greet; // signal to send events  
};
```

A Multithreaded, Event-Driven „Hello World“ Application

Step 3: Putting it all together

```
void printEvent( const ptv::Event& ev )
{
    if( ev.typeInfo() == HelloEvent::typeInfo() ) {
        std::cout << (const HelloEvent&)(ev).text() << std::endl;
    }
}

int main( int argc, char* argv[] )
{
    Application app;
    HelloThread ht;
    ptv::connect(ht.greet, app, &Application::commitEvent);
    ptv::connect(app.event, printEvent);
    ht.start();
    return app.run();
};
```

Observations

RTTI is a high-cost feature used at very low cost

- > Slow typeid() lookup happens only once, since static
- > Comparing two std::type_info objects is fast
- > Very flexible dispatching code possible

Threads are separated from each other

- > EventLoop is point of synchronisation
- > „Long events“ may block EventLoop

Gfx Module Overview

Basic drawing

- > Pen, Brush, Font
- > 2D drawing operations in Painter

Image processing

- > Images and SubImages
- > Colorspaces: ARGB, RGB24, RGB16, RGB15, CMYK, YUV...
- > Algorithms: greyscale() ...

Soon: Image I/O

- > PNG, JPEG ...

Gui Module Overview

Application GUI applications

- > Thread-safe handling of native GUI events and custom events

Widgets

- > Events for low-level events
- > Signals and Delegates for high-level events

Pixmapes

- > Double buffering

Extends Gfx drawing API

- > Painters for widgets and pixmaps

A „Hello World“ GUI Application

```
int main( int argc, char* argv[ ] )
{
    ptv::gui::Application app;

    ptv::gui::Button btn;
    btn.setText(„Hello World!“);
    btn.show();
    ptv::connect(btn.clicked, app, &ptv::gui::Application::exit);

    return app.run();
};
```

Gui Event handling

Typical GUI events

- > CloseEvent, MoveEvent, ResizeEvent, PaintEvent, MouseEvent, MouseMoveEvent, KeyEvent

GUI events are dispatched to virtual functions in Widget

- > Widget::paintEvent(const PaintEvent& ev)
- > Widget::resizeEvent(const ResizeEvent& ev)
- > Widget::mouseEvent(const MouseEvent& ev)
- > Can be overridden to refine a Widget

Example: Scribble

Let's write a small drawing program

- > Allows painting with the mouse
- > Implements double-buffering
- > Changes the pen color when the 'r', 'g' or 'b' key is pressed
- > Resizes correctly

```
class Scribble : public gui::Widget  
{  
    ...  
};
```

Catching Mouse Move Events

Draw line on mouse events

> Override Widget::_mouseMoveEvent

```
class Scribble : public gui::Widget {
    Point _last;
};

void Scribble::_mouseMoveEvent(const MouseMoveEvent& ev)
{
    if( event.modifiers() & MouseMoveEvent::LeftButtonDown ) {
        gui::Painter widgetPainter = this->painter();
        widgetPainter.drawLine( _last, Point( event.x(), event.y() ) );
    }
    _last = Point( event.x(), event.y() );
}
```

Double Buffering

Paint to Pixmap and Widget

```
class Scribble : public gui::Widget {
    Point _last;
    Pixmap _buffer;
};

void Scribble::_mouseMoveEvent(const MouseEvent& ev)
{
    if( event.modifiers() & MouseEvent::LeftButtonDown ) {
        gui::Painter widgetPainter = this->painter();
        gui::Painter pixmapPainter = _buffer.painter();
        widgetPainter.drawLine( _last, Point( event.x(), event.y() ) );
        pixmapPainter.drawLine( _last, Point( event.x(), event.y() ) );
    }
    _last = Point( event.x(), event.y() );
}
```

Double Buffering

Paint events are sent if parts of the widget needs updating

> Copy area of Pixmap to Widget

```
void Scribble::_paintEvent(const PaintEvent& event)
{
    gui::Painter widgetPainter = this->painter();
    widgetPainter.drawPixmap( event.origin(), _pixmap, event.rect() );
}
```

Resizing the drawing area

Resize events are sent if the Widget is resized

> Update backbuffer

```
void Scribble::_resizeEvent(const ResizeEvent& event)
{
    QPixmap tmp( event.width(), event.height() );

    // draw white background and copy old backbuffer
    gui::Painter painter = tmp.painter();
    painter.setBrush( gfx::Brush(ARgbColor(0xffff, 0xffff, 0xffff)) );
    painter.fillRect( gfx::Rect( Point(0, 0), newPixmap->size() ) );
    painter.drawPixmap( gfx::Point(0, 0), *_pixmap );

    _buffer = tmp;
}
```

Reacting to Key Events

Change Pen color on key events

> Override Widget::_keyEvent

```
void Scribble::_keyEvent(const KeyEvent& event)
{
    Pen pen; // black, width 1

    if( event.text() == 'r' )
        pen = Pen( 1, ARgbColor(0xffff, 0, 0) ); // red, width 1
    else if( event.text() == 'g' )
        pen = Pen( 1, ARgbColor(0, 0xffff, 0) ); // green, width 1
    else if( event.text() == 'b' )
        pen = Pen( 1, ARgbColor(0, 0, 0xffff) ); // blue, width 1

    this->painter().setPen(pen);
    _buffer.painter().setPen(pen);
}
```

Example: Scribble

Putting it all together

```
int main( int argc, char* argv[] )  
{  
    ptv::system::Application app;  
  
    Scribble scribble;  
    scribble.show();  
    ptv::connect(scribble.closed, app, &system::Application::exit);  
  
    return app.run();  
};
```

Erstklassig unterwegs. PTV.

Vielen Dank für Ihre Aufmerksamkeit



PTV Planung Transport Verkehr AG, 76131 Karlsruhe



© PTV AG / MAP&G